

Transport Industry Projects

A Portfolio of IT Solutions for Data Analytics & Software Development

Project Proposal

Transportation Security System

Prepared By: Ashley Chindanga
Date: August 2, 2026
Version: 1.0
Status: Draft
Project Type: Software Development & Computer Vision
Estimated Time: 4-6 Weeks

Project Code: TSS-2026-01

Confidential - For Review Only

Contents

1	Introduction	2
1.1	Background	2
1.2	Project Overview	2
1.3	Project Objectives	2
1.4	Target Audience	2
2	Problem Statement	3
2.1	Current Challenges	3
2.2	Why This Project Matters	3
3	Technical Scope	3
3.1	System Architecture	3
3.2	Core Modules	4
3.2.1	1. Data Collection Module	4
3.2.2	2. Driver Monitoring Module (Computer Vision)	4
3.2.3	3. Behavior Analysis Module	4
3.2.4	4. Alert and Notification Module	4
3.2.5	5. Dashboard Module	4
3.3	Technologies Used	5
3.4	Data Sources	5
4	Implementation Plan	5
4.1	Development Phases	5
4.2	Milestones	5
5	Deliverables	6
5.1	Code Repository	6
5.2	Documentation	6
5.3	Working Software	6
5.4	Portfolio Assets	6
6	Skills Demonstrated	7
7	Risk Assessment	7
8	Success Metrics	7
9	Conclusion	8
A	Appendix A: Sample Code Structure	9
B	Appendix B: Sample Dashboard Layout	9
C	Appendix C: References	10

1 Introduction

1.1 Background

The transportation and logistics industry is experiencing rapid digital transformation. With increasing concerns about driver safety, regulatory compliance, and operational efficiency, there is a growing demand for intelligent security systems that can monitor driver behavior and ensure safety on the roads.

According to the World Health Organization, approximately 1.3 million people die each year as a result of road traffic crashes, with driver fatigue and distraction being leading causes. This project addresses these critical safety challenges through technology.

1.2 Project Overview

The **Transportation Security System** is a comprehensive IT solution designed to monitor driver behavior, detect unsafe driving patterns, and prevent accidents before they occur. The system leverages:

- **Computer Vision** for driver fatigue and distraction detection
- **IoT Sensors** for vehicle telemetry data collection
- **Real-time Analytics** for immediate threat detection
- **Alert Systems** for proactive safety interventions

1.3 Project Objectives

1. Develop a real-time driver monitoring system using computer vision
2. Implement algorithms to detect unsafe driving behaviors
3. Create an automated alert system for safety violations
4. Build a comprehensive dashboard for safety analytics
5. Generate driver scorecards and safety reports

1.4 Target Audience

- **Fleet Managers** - Monitor driver performance across fleets
- **Transport Companies** - Ensure regulatory compliance
- **Safety Officers** - Track and improve safety metrics
- **Drivers** - Receive real-time feedback on driving behavior
- **Insurance Companies** - Access driving data for risk assessment

2 Problem Statement

2.1 Current Challenges

The transportation industry faces significant safety challenges:

Challenge	Impact
Driver Fatigue	20-30% of all road accidents
Distracted Driving	25% of all traffic accidents
Speeding	30% of fatal accidents
Harsh Braking	Vehicle wear, passenger discomfort
Lack of Real-time Monitoring	Delayed incident response

Table 1: Key Safety Challenges in Transportation

2.2 Why This Project Matters

- **Safety First:** Prevent accidents and save lives
- **Cost Reduction:** Reduce maintenance costs from harsh driving
- **Compliance:** Meet regulatory safety requirements
- **Insurance Benefits:** Lower premiums through documented safety
- **Driver Improvement:** Provide actionable feedback for drivers

3 Technical Scope

3.1 System Architecture

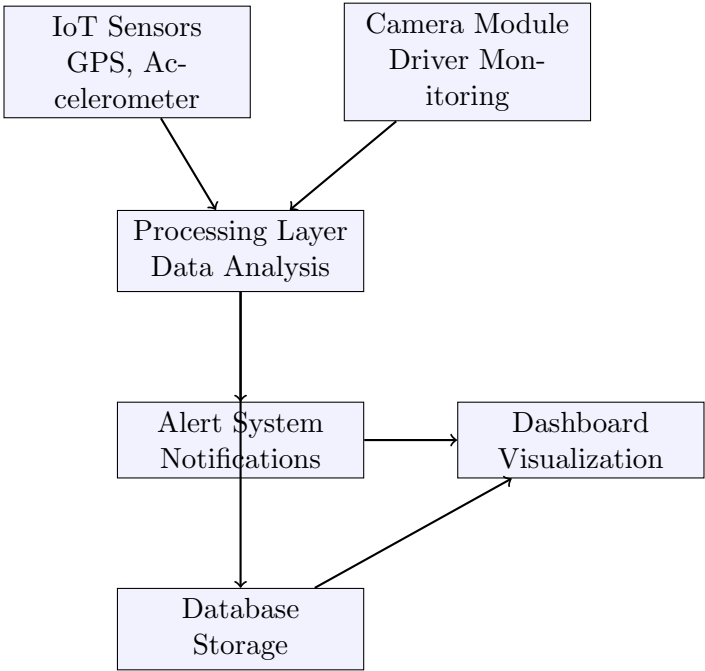


Figure 1: High-Level System Architecture

3.2 Core Modules

3.2.1 1. Data Collection Module

- GPS data collection (speed, location, route)
- Sensor data collection (acceleration, braking, cornering)
- Video feed capture (driver's face, cabin view)
- Real-time data streaming via MQTT/WebSockets

3.2.2 2. Driver Monitoring Module (Computer Vision)

- Facial landmark detection using Dlib/OpenCV
- Eye blink detection for fatigue assessment
- Head pose estimation for distraction detection
- Yawning detection as fatigue indicator
- Distracted driving detection (phone use, looking away)

3.2.3 3. Behavior Analysis Module

- Speeding detection (GPS speed vs. road limits)
- Harsh braking detection (deceleration $> 3 \text{ m/s}^2$)
- Harsh cornering detection (lateral acceleration $> 2 \text{ m/s}^2$)
- Driver scorecard generation (0-100 rating)
- Trend analysis and predictive insights

3.2.4 4. Alert and Notification Module

- Real-time alert generation for violations
- Multi-channel notifications (Email, SMS, Push)
- Escalation protocols for critical events
- Alert history logging and reporting

3.2.5 5. Dashboard Module

- Real-time driver monitoring interface
- Safety analytics and trend visualization
- Driver ranking leaderboard
- Incident reporting and management
- Custom report generation

3.3 Technologies Used

Category	Technology	Purpose
Programming	Python 3.9+	Core development
Computer Vision	OpenCV, Dlib	Face detection, tracking
Machine Learning	TensorFlow/PyTorch	Fatigue detection model
Web Framework	Flask/Dash	Dashboard application
Database	SQLite/PostgreSQL	Data storage
Real-time	WebSockets, MQTT	Data streaming
Visualization	Plotly, Matplotlib	Charts and graphs
Testing	Pytest, unittest	Unit testing
Version Control	Git, GitHub	Source code management

Table 2: Technology Stack

3.4 Data Sources

- **Synthetic Data:** Generated using simulators (initial phase)
- **Open Datasets:** UCI Driving Behavior Dataset, NTHSA Crash Data
- **Simulated IoT Data:** Custom GPS and sensor simulators
- **Webcam Feed:** Real-time video from test environment

4 Implementation Plan

4.1 Development Phases

Phase	Activities	Duration	Deliverables
Phase 1	Data collection setup, sensor simulation	Week 1	Working data pipeline
Phase 2	Computer vision implementation	Week 2-3	Fatigue detection working
Phase 3	Behavior analysis algorithms	Week 3-4	All detectors functional
Phase 4	Alert system and dashboard	Week 4-5	Full interface complete
Phase 5	Testing, integration, documentation	Week 5-6	Complete system ready

Table 3: Development Timeline

4.2 Milestones

1. **Week 1:** Setup project structure and data pipeline
2. **Week 2:** Face detection and tracking working
3. **Week 3:** Fatigue and distraction detection functional
4. **Week 4:** All behavior detectors implemented
5. **Week 5:** Dashboard and alerts fully functional
6. **Week 6:** System testing and documentation complete

5 Deliverables

5.1 Code Repository

- Complete source code on GitHub
- Well-structured project with clear documentation
- Modular code with separation of concerns
- Comprehensive README with setup instructions

5.2 Documentation

- Project proposal (this document)
- Technical requirements specification
- System architecture document
- API documentation
- User guide and installation manual

5.3 Working Software

- Real-time driver monitoring system
- Web-based dashboard for safety analytics
- Automated alert system
- Driver scorecard and reporting system

5.4 Portfolio Assets

- Demo video (5 minutes)
- Screenshots of working system
- Presentation slides for job interviews
- GitHub repository with detailed README

6 Skills Demonstrated

Skill Category	Specific Skills
Programming	Python, Object-Oriented Design, API Development
Computer Vision	OpenCV, Dlib, Face Detection, Image Processing
Data Analytics	Real-time Processing, Statistical Analysis
IoT	Sensor Data Processing, MQTT Protocol
Web Development	Flask/Dash, HTML/CSS, JavaScript
Database	SQL, Data Modeling, Query Optimization
Machine Learning	Classification Models, Model Training
Visualization	Plotly, Matplotlib, Dashboards
Soft Skills	Documentation, Project Management, Problem Solving

Table 4: Skills Demonstrated by This Project

7 Risk Assessment

Risk	Impact	Mitigation
Hardware access limitation	Development delay	Use simulators initially
Model accuracy issues	False positives	Use thresholds, continuous tuning
Performance issues	Slow processing	Optimize algorithms, use caching
Integration challenges	Delayed timeline	Regular integration testing
Data privacy concerns	Compliance issues	Anonymize data, local processing

Table 5: Risk Assessment and Mitigation

8 Success Metrics

The project will be considered successful if we achieve:

- Accuracy:** Fatigue detection with ≥85% accuracy
- Speed:** Real-time processing (≤500ms per frame)
- Functionality:** All 5 core modules fully functional
- Usability:** Dashboard intuitive and informative
- Documentation:** Complete and well-organized
- Code Quality:** PEP8 compliance, modular design

9 Conclusion

The Transportation Security System is a comprehensive IT solution that addresses critical safety challenges in the transportation industry. By leveraging computer vision, IoT sensors, and real-time analytics, this system will demonstrate the ability to:

- Build intelligent monitoring systems
- Apply computer vision to real-world problems
- Develop real-time analytics applications
- Create user-friendly safety dashboards
- Demonstrate industry-relevant technical skills

This project not only creates a valuable safety solution but also builds a strong portfolio demonstrating expertise in computer vision, IoT, data analytics, and software development - exactly what employers in the transportation technology sector are looking for.

A Appendix A: Sample Code Structure

Listing 1: Sample Structure for Fatigue Detection

```
1 import cv2
2 import dlib
3 from scipy.spatial import distance
4
5 class FatigueDetector:
6     def __init__(self):
7         self.detector = dlib.get_frontal_face_detector()
8         self.predictor = dlib.shape_predictor('
9             shape_predictor_68_face_landmarks.dat')
10        self.eye_aspect_ratio_threshold = 0.25
11
12    def detect_fatigue(self, frame):
13        """Detects fatigue based on eye closure"""
14        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
15        faces = self.detector(gray)
16
17        for face in faces:
18            landmarks = self.predictor(gray, face)
19            left_eye = self._get_eye_landmarks(landmarks, 'left')
20            right_eye = self._get_eye_landmarks(landmarks, 'right')
21
22            ear_left = self._calculate_eye_aspect_ratio(left_eye)
23            ear_right = self._calculate_eye_aspect_ratio(right_eye)
24            ear_avg = (ear_left + ear_right) / 2.0
25
26            if ear_avg < self.eye_aspect_ratio_threshold:
27                return True # Fatigue detected
28        return False
```

B Appendix B: Sample Dashboard Layout

Listing 2: Dashboard UI Structure

```
1 # Flask App Structure
2 @app.route('/')
3 def dashboard():
4     return render_template('dashboard.html',
5         driver_data=get_driver_data(),
6         alerts=get_active_alerts(),
7         stats=get_safety_stats()
8     )
9
10 # Dashboard Components
11 - Real-time speed gauge
12 - Fatigue status indicator
13 - Alert feed
14 - Driver score card
15 - Trend charts
16 - Leaderboard
```

C Appendix C: References

1. OpenCV Documentation: <https://docs.opencv.org/>
2. Dlib Documentation: <http://dlib.net/>
3. WHO Road Safety Report: <https://www.who.int/roadsafety>
4. NHTSA Crash Data: <https://www.nhtsa.gov/>
5. Flask Documentation: <https://flask.palletsprojects.com/>